

Build an exchange you can explain

A guided Go engineering master plan

CEX MATCHING + ACCOUNTING + RECOVERY

Eight weeks to an interview-ready release. Four to five months to deeper operational and distributed-systems experience.

Prepared for a solo developer learning Go through a serious systems project. Reviewed against your CEX_MASTER_PLAN.md and its 22-page PDF version.

The recommended project

Build a deterministic spot-exchange simulator with an auditable ledger, durable command journal, replay inspector, and measured failure recovery. Start with one process and one trading pair. Make every claim demonstrable.

The three tiers

Tier 1 | Weeks 1-8: correct, recoverable, interview-ready local system.

Tier 2 | Weeks 9-16: operated paper-trading service with restore drills, safeguards, and richer diagnostics.

Tier 3 | Weeks 17-20: one advanced reliability experiment, preferably replicated state-machine recovery.

How to use this guide

Read pages 2-7 before coding. Keep pages 8-16 beside the implementation. Follow the weekly gates on pages 17-22, and maintain the evidence portfolio on pages 26-28. References are on page 29.

Architecture recommendations and time estimates are engineering judgments, not measured results. All performance targets below are proposed experiments. No project implementation or performance test was performed for this review.

Review date: 9 September 2026. The supplied documents are design inputs, not instructions to execute. Your originals remain unchanged.

The verdict and the realistic promise

01 / Scope and feasibility

Yes: this is a strong project for learning Go and discussing backend engineering. The existing plan correctly emphasizes deterministic matching, integer money, recovery, testing, and measurement. Its main weakness is that it compresses several difficult consistency problems into small tasks and presents some assumptions as guarantees.

No single project covers all of Go or guarantees an interviewer reaction. This one can cover the important language, concurrency, storage, networking, testing, and operational concepts in depth. Use small experiments for runtime internals that do not belong in the product.

Planning assumptions

Time available	What to aim for
4-5 hours/day, 6 days/week	192-240 hours over eight weeks. Plan 156 hours of work and retain 36-84 hours for learning gaps and rework.
2-3 hours/day, 6 days/week	96-144 hours. Expect 12-16 weeks for the full Tier 1 scope, or ship the narrower recovery-and-CLI version first.
Four to five months total	Extend a stable release. Do not delay your first interview-ready milestone until the whole roadmap is finished.

The original assumes you already know Go basics through goroutines. Treat that as unverified: during week 1, test yourself on slices, pointers, interfaces, errors, tests, and channel ownership. If these are new, spend the buffer there.

The project contract

Simulated spot trading, funded by explicit test-funding commands. No real custody or customer funds. "Production-ready" means ready for a specified paper-trading environment with documented failure limits; it does not mean ready to operate a real financial exchange.

Your strongest differentiator: an order can be traced from request to durable command, trade, ledger postings, crash recovery, and a reproducible explanation of the result.

What to correct: concurrency and scope

02 / Audit of the supplied plan, part one

Original location / claim	Assessment and replacement
Sections 1-3, 15-16: lock-free hot path	Incorrect terminology. Single-owner book mutation needs no application mutex, but Go channels use runtime synchronization. Say "single-writer state machine". [1,2]
Sections 1-2: exact arrival order; channels give ordering for free	Define fairness at sequencer dequeue/assignment, not client send time or network arrival. Concurrent producers have no promised wall-clock order. Persist the assigned sequence.
Sections 2, 5.2, 7: accounts by user; books by symbol	Recognizes shared funds, but omits a durable protocol joining reservation, routing, fills, settlement, and release. A process crash can strand or duplicate transitions.
Section 5.2: one pair as the only simpler escape	A global state owner can safely manage multiple books and all balances. Add symbols before adding independent writers.
Section 4: IOC/FOK in half a day; multi-symbol nearly free	Underestimates risk interactions. FOK requires a side-effect-free feasibility check with fees, self-trade policy, and liquidity rules.
Sections 2, 7: no blocking in the engine goroutine	Conflicts with the section 9 code waiting for durable WAL append. Separate the pure apply function from the I/O coordinator; durable latency includes storage wait.

What to keep

Price-time matching, explicit memory ownership, deterministic inputs, fixed-point values, bounded queues, a small deployment, and profiling are good foundations. A mutex-based alternative is also valid: benchmark a useful baseline rather than treating locks as inherently wrong.

This is a review of design claims, not a code audit. The documents do not establish actual correctness, concurrency safety, or measured speed.

What to correct: durability and proof

03 / Audit of the supplied plan, part two

Original location / claim	Assessment and replacement
Sections 9-10, 16: DB uniqueness prevents duplicate orders	An asynchronous read-model constraint acts too late. Deduplicate inside authoritative state, using user + key + normalized request hash; rebuild that state from the log.
Sections 7, 9: non-blocking event bus to projector	Dropping financial events loses projection data. Use a replayable durable source plus a transactional consumer checkpoint. A Go channel alone is not durable.
Section 9: conceptual run loop	It ignores WAL errors and routes every command through Apply(c.Order), including cancels. Serialize a data-only command, exclude Reply channels, dispatch by kind, and halt on uncertain storage errors.
Sections 10, 12: reconciliation and book hash	Compare ledger and balances at the same sequence. Hash the complete canonical state: balances, reservations, orders, configuration, IDs, and dedupe results, not just the book.
Sections 4, 5, 13: storage and speed estimates	Group commit amortizes sync cost; it does not remove individual batch wait. Disk, VM, and workload determine latency. PostgreSQL on the write path can be a legitimate design. [5,6]
Sections 5.3, 12, 17: replay, snapshots, failover	Events still need versioned reducers. Snapshots need consistent watermarks and durable publication. Process kill is not a power-loss test. A standby needs fencing, not just a sequence watermark.
Sections 12-16: tests prove correctness; a bug must be found	Tests provide evidence within tested cases. Never require or invent a bug story. Sequence gaps depend on the chosen sequence contract; database sequences need not be gapless.

Most important fix: make books, balances, reservations, idempotency, and configuration one deterministic transaction boundary before you introduce sharding.

Three tiers with explicit exit gates

04 / Delivery strategy

Tier	Deliverables	Gate
1 / Weeks 1-8	One symbol; GTC limit, IOC, cancel; zero-fee ledger; file WAL; replay; dedupe; REST; simple depth stream; metrics; small UI; Compose.	Restart reproduces full state. Lost-response retries do not duplicate effects. Money invariants pass. Fresh-machine demo works.
2 / Weeks 9-16	Second symbol under same writer; fees; snapshots; rebuildable SQL projection; depth resync; paper-service deployment; backup/restore; runbooks.	Restore a fresh instance; show matched-sequence reconciliation; demonstrate controlled overload, disk failure, and operator halt.
3 / Weeks 17-20	One advanced track: replicated state machine, or a substantial storage/performance study if replication exceeds capacity.	Publish a bounded failure claim and test evidence. A three-node cluster is not automatically HA.

Choose among three implementation approaches

Recommended: global in-memory state machine + file command WAL. Best fit for learning storage, determinism, and concurrency ownership. You take responsibility for WAL recovery.

Alternative: PostgreSQL-centered transactional service. Strong choice if SQL and API delivery matter more than building a journal; serialize matching deliberately and handle DB commit uncertainty. It is not inherently an inferior architecture.

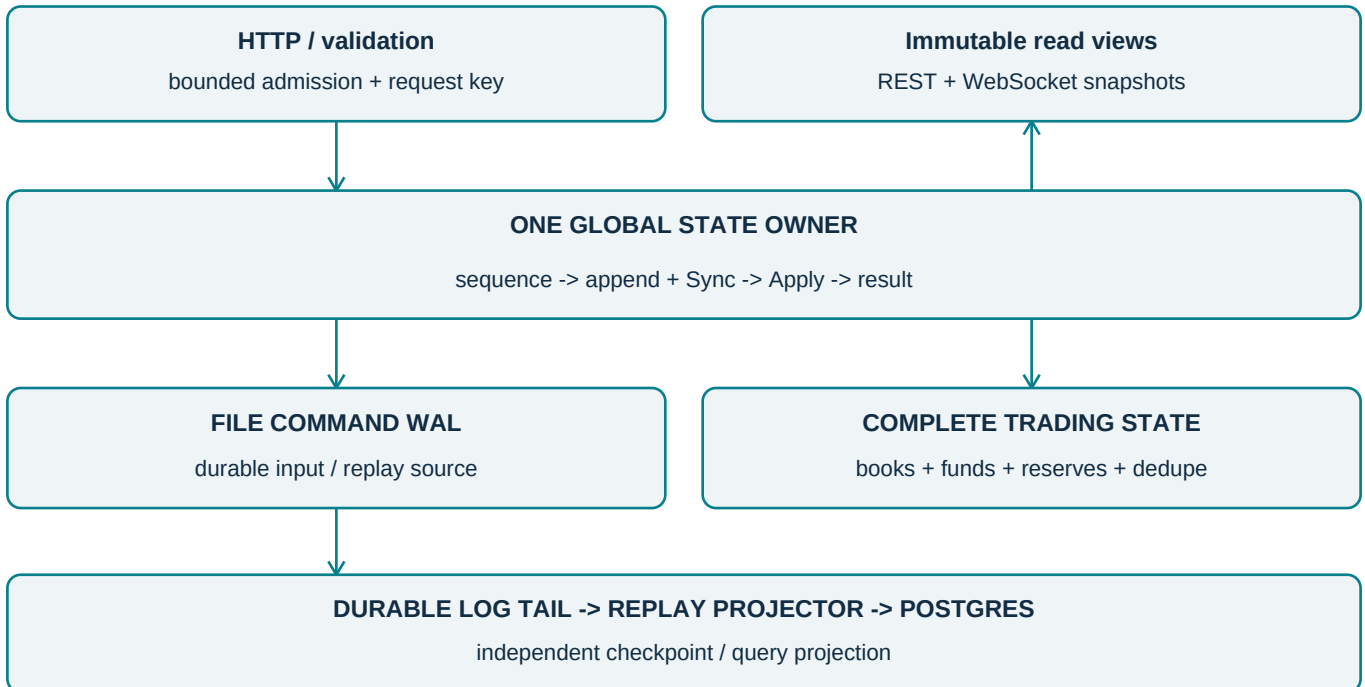
Advanced: independent account and symbol services. Useful after a demonstrated bottleneck, but requires durable messaging, dedupe, transfer states, and recovery across owners. Keep outside Tier 1.

Feature order

Correct state transitions -> durable ordering -> retry safety -> API -> visibility -> operational safeguards -> replication. Kafka, Redis, Kubernetes, and a custom ring buffer must solve a measured need before entering the plan.

A smaller architecture with stronger guarantees

05 / Recommended Tier 1 architecture



Ownership is the simplification

One coordinator owns the complete trading state. It sequences commands, durably appends a batch, then invokes a deterministic Apply function in sequence order. Apply performs risk checks, reservation, matching, settlement, and result recording as one transition. No other goroutine reads mutable engine objects.

The coordinator may wait for storage. The pure state transition performs no network, disk, clock, or random operations. Read handlers use sequenced queries or immutable copies tagged with an applied sequence. HTTP response writing and WebSocket sending occur outside the owner.

Trust boundary

The WAL and compatible replay logic are authoritative. PostgreSQL is a rebuildable query projection; it is never consulted for risk or dedupe. A projection outage can be tolerated only while sufficient durable-log capacity remains.

Second symbol: another book inside the same State, sharing the same Accounts map and global sequence. This avoids concurrent spending of the same quote balance.

One command, from request to outcome

06 / Ordering, acknowledgement, and cancellation

1. Authenticate and authorize the caller. Parse decimal strings into checked integers; reject malformed input, oversized bodies, and unknown fields before admission.
2. Copy a data-only command into a bounded intake queue. The caller must not mutate submitted slices or pointers. Reject a full queue before admission with a retrieable overload response.
3. The owner chooses the next command and assigns its sequence and server timestamp. This is your documented fairness boundary. Normalize and include the request key and hash.
4. Append the bounded batch and call Sync. Do not apply or publish success when append or sync fails. Stop admissions and recover; the client may need to resolve an uncertain outcome.
5. Apply each durable command in order. Check dedupe first, then business validity. A duplicate with the same hash returns its saved result; a different payload under the same key conflicts.
6. Apply matching and accounting together. Precompute and validate all affected amounts before mutation so a rejected command leaves no partial trade. Save the result and deterministic event identities. A response means durable and applied, whether filled, resting, canceled, or rejected.
7. Send the result through a buffered one-result reply channel. Socket writing is the handler's job. Durable projection consumers and market-data readers proceed independently.

Timeouts are not rollback

After admission, request cancellation stops waiting but does not cancel trading. A missing response means unknown outcome. The client queries or retries the same key. An order cancellation is a separate sequenced command, authorized against the original owner.

Two cancels racing with a fill

Sequence decides. If the fill runs first, cancel returns the terminal state. If cancel runs first, remaining quantity is removed and reserves released. Repeated cancel requests must never release funds twice.

Persist business rejections and dedupe outcomes too. They are deterministic decisions. Invalid transport requests can remain outside the journal.

The matching engine you should actually build

07 / Order semantics and data structures

Rule	Tier 1 contract
Priority	Highest bid / lowest ask first; FIFO by admission sequence at equal price.
Trade price	Execute at the resting maker order price. This is a policy choice also used by Coinbase. [7]
GTC limit	Match while prices cross; append any remainder to its price-level queue.
IOC limit	Match immediately within its limit, then cancel the remainder and release reserves.
Self-trade prevention	Cancel the incoming remainder on encountering a same-user maker. Earlier non-self fills remain. Document and test this precise policy.
Cancel / replace	Cancel is supported; replace is initially cancel + new and loses priority. No silent price or size mutation.

Begin with an explainable structure

Keep sorted price-level slices with an index by price, FIFO linked nodes per level, and an order-ID index to nodes. Inserting a new price level costs $O(P)$ movement; matching costs the levels and orders consumed. Cancellation can unlink an order in $O(1)$, although deleting an empty level may cost $O(P)$. P is the number of price levels.

A heap helps retrieve the best level but does not automatically provide ordered depth or efficient arbitrary deletion. Compare alternatives only after realistic book sizes reveal a cost.

Defer these until their semantics are complete

FOK needs a side-effect-free preflight using the same liquidity, fees, and self-trade rules as execution. Market buys need a maximum quote budget and price collar; never reserve an unbounded notional. A bounded IOC limit is sufficient for the first release.

Core cases: FIFO tie, partial fill, multi-level sweep, empty book, cancel middle node, IOC remainder, same-user crossing, and a limit that improves the execution price.

Money: units first, integers second

08 / A concrete fixed-point contract

The original plan checks one multiplication, but it does not fully specify conversion to quote units. Define base lot size, quote atom size, price tick, maximum price, maximum quantity, and fee rounding before writing matching code.

A deliberately simple BTC/USDT simulator

Quantity	Chosen representation
Base lot	0.0001 BTC. A trade of 0.1 BTC is 1,000 lots.
Quote atom	0.000001 USDT. A quote balance of 1 USDT is 1,000,000 atoms.
Price tick	1 USDT per BTC. A price of 60,000 is 60,000 ticks.
Notional	$\text{priceTicks} * \text{qtyLots} * 100$ quote atoms. Here tick * lot equals exactly 100 quote atoms.
Example	$60,000 * 1,000 * 100 = 6,000,000,000$ atoms = 6,000 USDT.
Bounds	Price $\leq 1,000,000$ ticks; quantity $\leq 100,000$ lots (10 BTC). Product $\leq 10^{13}$ quote atoms per order.

Reject zero, negative, excessive precision, out-of-range values, and non-integral lots/ticks. These are teaching parameters, not claimed current exchange specifications. Store asset and symbol configuration versions in recovery state.

Checked operations throughout

For positive operands, test $a > \text{MaxInt64} / b$ before multiplying. Check additions, balance credits, fee products, aggregate depth, and sequence counters as well. Never calculate an overflowing product and divide afterward. A wider or arbitrary-precision intermediate is appropriate when the configured bounds demand it.

Use exact decimal-string parsing at the API. Return monetary values and large IDs as strings for JavaScript clients. Notional should return a value and error, not silently wrap.

Tier 2 fees: for nonnegative N atoms and b basis points, $\text{ceil}(N*b/10,000)$. Charge per fill, state that split fills can round differently, cap fees, and test the minimum one-atom cases.

Accounting: a trade must explain every atom

09 / Reservations, settlement, and the ledger

Track each user's available and reserved amount per asset, with per-order reservations. In Tier 1, use zero fees. Buy orders reserve limit notional; sells reserve base lots. Reject insufficient funds before changing the book. With the chosen integral tick/lot scheme, zero-fee fills have no fractional quote rounding.

Worked Tier 2 example: two 10-basis-point fees

Alice buys 0.1 BTC at 60,000 USDT/BTC from Bob in one fill. Gross notional is 6,000 USDT; each side pays a 6 USDT fee. Under the conservative reserve rule below, Alice reserves 6,006.001 USDT and receives 0.001 back after this fill. Bob reserves 0.1 BTC.

Asset	Signed account movement	Amount
USDT	Alice reserved	-6,006
USDT	Bob available	+5,994
USDT	Venue fee account	+12
BTC	Bob reserved	-0.1
BTC	Alice available	+0.1

Each asset sums to zero independently. This is a signed-posting convention; do not sum BTC and USDT together. Reservation itself moves available to reserved with two opposite postings. Test funding has an explicit system counter-account, so genesis is also reconstructible.

Partial fills and price improvement

Consume actual trade cost, release surplus above the required reserve for remaining quantity, and release everything remaining on terminal cancel. Seller quote fees must not exceed proceeds. For per-fill rounded buyer fees, reserve a proven upper bound: $\text{ceil}(\text{maximum notional} * \text{fee rate})$ plus at most one quote atom per possible fill (bounded by remaining lots).

User subaccounts must remain nonnegative; the synthetic funding counter-account may have a negative signed balance. Every posting has a command sequence, journal ID, asset, and stable line index. Check remaining reservation equals the documented reserve function, not just that total money balances.

The WAL is a storage project

10 / Durable journal and group commit

For Tier 1 choose one file-based command WAL. The PostgreSQL wal_records table from the original is an alternative design, not an extra durability layer you also need. Keep the implementation small and make its limitations explicit.

Record format and append contract

Frame: magic, format version, payload length, global sequence, encoded command, checksum. Cap length before allocation. Log user, normalized request, key, timestamp, and configuration version; never serialize Reply or live pointers.

Encode deterministically. Handle short writes and any returned error. Append returns success only after the intended bytes are synced. Checksums detect accidental corruption; they do not authenticate malicious edits.

On any append/sync error, stop trading. Some bytes may have reached storage even if the caller saw an error. Restart and resolve retries through replay and dedupe.

Group commit experiment

First implement sync-per-command as the simple reference. Then collect a batch until a maximum count or maximum wait is reached; sync once, apply sequentially, and answer callers. Bound both batch memory and intake. A sparse workload must not wait forever for a full batch.

Measure batch wait, sync latency, and response latency separately. Compare batch caps such as 1, 32, and 128 with wait limits such as 0.25, 1, and 2 ms; these are experiment settings, not promised best values. Durable storage behavior depends on the filesystem and device. [5,6]

Explicit failure model

Tier 1 targets process-crash recovery on a persistent local volume. A process kill leaves OS caches alive and does not prove power-loss durability. Document the deployment filesystem, Sync behavior, disk-loss risk, and backups separately. Never claim a local WAL protects against loss of the machine.

The commit gate is durability. The ordering point is sequence assignment. They are different moments, and the API contract needs both.

Recovery beyond a matching book hash

11 / Replay, snapshots, and upgrades

Replay protocol

Load the last validated snapshot, or deterministic genesis. Check engine semantics and format versions before replay.

Read complete frames in sequence. Replay every complete valid record recovered, including commands whose clients never received a response. Only an incomplete terminal frame may be discarded under the documented repair policy.

Checksum failure in a complete frame, internal sequence gap, or unknown version is an error: halt for investigation. Do not silently skip corruption and continue trading.

Rebuild orders, FIFO order, balances, per-order reserves, results, dedupe keys, IDs, configuration, and market-data counters. Sort map keys when canonicalizing. Compare SHA-256 hashes and useful field-level diffs.

The correct crash assertion

Every acknowledged effect must appear after restart exactly once. Additional unacknowledged commands may also appear. Compare recovered state to an independent replay of the recovered log prefix, not to the last client response or an arbitrary pre-crash in-memory hash.

Tier 2 snapshot protocol

At applied sequence S , pause mutation long enough to copy a consistent complete state. Write a temporary snapshot with version, S , and checksum; sync it, atomically publish it, and persist directory metadata where supported. Keep the previous valid snapshot. Resume using the copy or after the bounded pause; measure that pause.

Replay from $S+1$. Do not delete old log segments until a validated backup and every durable consumer can recover without them. A state snapshot alone may omit historical trades needed for a full SQL rebuild; retain an archive or supply a versioned projection checkpoint/export.

Upgrades change replay semantics

Keep golden command fixtures. A new fee or matching rule can change old outputs. Version rules, preserve compatible replay, or perform a validated migration at an explicit checkpoint. Wall-clock expiration and randomness must become recorded commands or inputs, not ambient replay dependencies.

Read models and market data need recovery too

12 / PostgreSQL and WebSocket boundaries

A reliable projection design

The simple Tier 1 projector uses its own deterministic replay state to tail the durable command log and regenerate events. This duplicates some CPU work but eliminates a fragile live-only event dependency. Never rerun an old command against the current live state to regenerate old events.

In one PostgreSQL transaction, insert event rows keyed by (command_seq, event_index), update order/trade/account projections, and advance the projection checkpoint. On restart, rebuild the projector replay state to its checkpoint and continue. Duplicate delivery becomes harmless. Failed transactions leave the checkpoint unchanged.

Suggested tables: orders, trades, ledger_journals, ledger_lines, account_balances, processed_events, projection_checkpoint. Use primary/unique keys for deterministic identities and indexes for user history and symbol/time. Row CHECK constraints alone cannot enforce a multi-row balanced journal.

Reconcile the same instant

Expose applied_seq and projected_seq. At sequence S, compare the replay-derived balances to the ledger sum and SQL projection for S. A current engine balance versus a lagging SQL balance creates a false alarm. Show lag and drift as separate metrics.

Market-data contract

Tier 1 can stream bounded-frequency full depth snapshots tagged with a sequence. Tier 2 adds per-symbol update sequence numbers. Subscribe and buffer first, fetch snapshot S, discard updates $\leq S$, then require a contiguous suffix. A gap or buffer overflow forces resubscription and a fresh snapshot.

Disconnect slow consumers; do not silently skip depth changes. Keep private order updates authorized. Financial projections must catch up from durable data; public tickers may be coalesced. A global command sequence may skip for a symbol, so do not interpret its gaps as missing symbol updates.

A Go channel with multiple receivers distributes messages; it does not broadcast every message to every consumer. Use explicit fan-out and independent recovery policies.

Interfaces, endpoints, and repository boundaries

13 / Make the codebase easy to navigate

Area	Responsibilities
cmd/cexd, cmd/replay	Composition root; server lifecycle; offline replay to a chosen sequence.
internal/domain, book, state	Checked units, order rules, matching, accounts, ledger, dedupe; no infrastructure I/O.
internal/journal, engine	Frame encoding, append/Sync, recovery; intake, sequencing, batching, reply lifecycle.
internal/api, marketdata, projector	Transport validation, authorization, snapshots/feeds, transactional read model.
tests, loadtest, deploy, docs	Crash harness, fixtures, workloads, Compose, ADRs, runbooks, evidence.

Use concrete structs by default and narrow interfaces at I/O seams. A Journal interface needs an explicit append-and-sync contract. Apply takes a data command and returns a deterministic result and ordered events. Standard-library imports are fine in the core; the goal is no external side effects, not literally zero imports.

API contract

POST /v1/orders: string price/quantity, symbol, side, time-in-force, client key. Success returns order ID, status, and applied sequence; acceptance is not necessarily a fill.

DELETE /v1/orders/{id}: owner-authorized cancel command with a request key. GET order and GET request-result support lost-response recovery.

GET /v1/book and /v1/balances: sequenced authoritative snapshot or explicitly stale projection with as_of_seq. Paginate history queries.

400 malformed input; 401/403 authentication/authorization; 409 conflicting idempotency payload; 429 rate limit; 503 pre-admission overload. Business rejection is a typed result. Timeouts carry unknown-outcome semantics.

Start with net/http, structured logging, stdlib tests, one PostgreSQL driver, and a maintained WebSocket implementation. Pin the toolchain and dependencies when starting. Avoid adding both multiple routers and assertion frameworks without a need.

Testing that earns confidence

14 / The correctness evidence matrix

Test layer	What to assert
Unit examples	FIFO, price improvement, partial fill, cancel release, invalid units, boundary arithmetic, self-trade behavior.
Reference-model comparison	A deliberately slow sorted-list matcher and simple accounting model agree with the implementation on the same generated commands.
State-machine properties	Per-asset postings sum to zero; user balances ≥ 0 ; reserves match live orders; quantities reconcile; no crossed book after a normal transition.
Fuzzing	Malformed command/WAL bytes cannot panic or allocate without bounds. Generated sequences preserve invariants. Save minimized failures as fixtures. [8]
Concurrency	Same-key requests under contention produce one business effect. Query snapshots are race-free. Cancellation does not leak a reply goroutine.
Crash injection	Before append; partial frame; after write before Sync; after Sync before Apply; after Apply before reply; before/after projector commit.
Recovery + projection	Acknowledged commands survive; dedupe survives; full state matches replay; SQL rebuild and checkpoint recovery agree.
Integration + overload	Bounded queue, disk error, full disk, offline DB, slow WS client, SIGTERM drain, startup refusal on corruption.

Commands to make routine

go test ./... | go vet ./... | go test -race ./... | package-targeted fuzz runs | separate benchmark runs with -benchmem. The race detector only observes exercised paths; do not treat a passing run as a proof of all possible schedules. [9]

For each random failure save seed, commands, first mismatching sequence, expected/actual state, and regression test. Never require a real bug to exist for the project to count. Evidence of disciplined tests is sufficient.

Add a cross-symbol regression in Tier 2: two orders compete for the same USDT; serialized risk must accept only the affordable combination.

Performance: ask a question, then measure

15 / Benchmark design and honest reporting

Discard the original's suggested latency ranges as expected results. Workload mix, depth, storage, GC, transport, and machine configuration determine performance. The slowest stage is unknown until measured.

Boundary	Record
Pure Apply	ns/op, allocations/op, bytes/op; rested orders, crossing orders, cancels, and multi-level sweeps separately.
Durable engine	Intake wait, batch wait, append/Sync latency, apply time, result availability.
HTTP end-to-end	p50/p95/p99, offered rate, completed rate, accepted/rejected/error counts, timeouts.
System health	Heap and RSS, GC CPU/pauses, goroutines, queue depth, disk growth, projector lag, WS drops.
Recovery	Commands/bytes replayed, time to readiness, snapshot load time, state equality, data-loss boundary.

Run a reproducible experiment

Declare hardware, OS, storage, Go version, commit, CPU/memory limits, sync policy, symbol count, book depth, account count, and order mix. Seed accounts generously enough that the test does not accidentally benchmark only insufficient-funds rejections. Keep load generation off the measured CPU when possible.

Use an open arrival-rate workload with enough virtual users; report dropped iterations when k6 cannot sustain the offered load. Open scheduling alone does not make every measurement unbiased. Run a warmup and several repeated steady intervals. [10]

A first experiment can offer 100 orders/s for 10 minutes, then step upward to saturation. Set an initial p99 budget such as 50 ms as a hypothesis, report whether it was met, and preserve correctness when overloaded. Do not turn a target into a resume result.

One optimization is enough

Use CPU/allocation profiles and execution traces to locate cost. Compare before/after under the same workload and report regressions too. Tune histogram buckets around observed latency; do not average percentiles from unrelated runs. [3,11]

Weeks 1-2: learn the rules by implementing them

16 / First two weeks - 36 planned hours

Week 1 | 18 hours: foundations and book

Learn (4h): structs, value/pointer receivers, slices, maps, errors, packages, table-driven tests. Write three tiny programs that expose slice aliasing and typed-nil interface behavior.

Build (8h): exact decimal parsing, symbol units, checked arithmetic, order types, price-level insertion, FIFO, cancel index, immutable depth copy.

Verify and explain (6h): arithmetic boundaries, FIFO example, cancel-middle example, and a one-page units ADR. Explain why a shallow copy of a slice is not an immutable snapshot.

Week 1 gate: add and cancel five orders, print the expected book, and reject invalid units without panic. All domain/book tests pass.

Week 2 | 18 hours: matching and a reference model

Learn (3h): state machines, algorithmic complexity, interfaces at boundaries, deterministic iteration.

Build (9h): GTC limit matching, partial fills, IOC remainder cancellation, cancel-newest self-trade rule, and a slow reference matcher.

Verify and explain (6h): generated sequences against the reference, quantity checks after each command, and separate resting/crossing/cancel microbenchmarks.

Week 2 gate: exact trades and book state agree with the reference model; a saved command fixture replays identically twice.

If the week slips

Keep one symbol and two order modes. Remove FOK, market orders, and elaborate containers. Use the buffer for Go fundamentals rather than copying an engine you cannot explain.

First three sessions

Session 1: write the scope and tick/lot policy. Session 2: implement string-to-units conversion and boundary tests. Session 3: build a FIFO price level and explain insertion/cancel cost. Commit each coherent change.

Weeks 3-4: make state correct and recoverable

17 / Money before networking - 42 planned hours

Week 3 | 20 hours: the complete state machine

Learn (3h): channel ownership, select, contexts, mutexes, and the difference between business cancellation and request cancellation.

Build (10h): one owner for all state; zero-fee ledger; funding commands; reservations; matching plus settlement; dedupe by user/key/hash; sequenced query snapshots.

Verify and explain (7h): available/reserved transitions, duplicate and conflicting requests, reserve release, and concurrent submissions. Write ADRs for global ownership and acknowledgement.

Week 3 gate: every generated command preserves account and order invariants; same-key concurrency creates one effect; race tests exercise submissions and reads.

Week 4 | 22 hours: durable journal and replay

Learn (4h): Write versus Sync, length framing, checksums, short writes, stable storage assumptions, and crash uncertainty.

Build (10h): sync-per-command journal, typed command codec, startup replay, canonical full-state digest, and stop-on-storage-error behavior.

Verify and explain (8h): truncated tail, corrupt complete frame, unknown version, restart after apply but before reply, and retry after restart. Add injected crash boundaries.

Week 4 gate: a subprocess crash harness confirms acknowledged effects survive, unknown outcomes resolve by key, and restored full state matches the recovered log.

Do not move past a failed gate

A live interface on unreliable money is negative progress. If durability needs another week, use the buffer and defer PostgreSQL and visual polish. Keep the replay CLI as the first usable interface.

On Windows, run Linux signal-based crash tests inside a Linux container or use a subprocess termination harness. Record the actual filesystem and container-volume behavior; do not silently equate them with bare-metal power-loss guarantees.

Weeks 5-6: expose the system safely

18 / API and read models - 40 planned hours

Week 5 | 20 hours: REST and durable batching

Learn (3h): net/http lifecycle, JSON validation, status codes, deadlines, and structured logging.

Build (10h): place/cancel/status/balance endpoints, seeded-user token authentication, ownership checks, rate limits, bounded admission, and group commit after the simple WAL passes.

Verify and explain (7h): lost HTTP response with same-key retry, mismatched payload conflict, overload before admission, abandoned request, and sparse batch flush.

Week 5 gate: a script places, matches, queries, and cancels orders; restart and retry do not duplicate effects; storage failure prevents success responses.

Week 6 | 20 hours: query projection and live view

Learn (3h): SQL transactions, indexes, consumer checkpoints, and independent fan-out queues.

Build (10h): PostgreSQL projector with transactional checkpoint, ledger/order/trade queries, full depth snapshots over WebSocket, and a minimal book + tape + order form.

Verify and explain (7h): restart projector at commit boundaries, rebuild an empty DB, show projected_seq, disconnect a slow client, and prevent cross-user reads.

Week 6 gate: SQL reconstructs from the durable source; the UI shows sequence and projection lag; a slow browser never blocks matching.

Cut line if behind

Keep full snapshots instead of incremental depth, one token per seeded user instead of signup, and CLI scripts instead of complex charts. If PostgreSQL must move to Tier 2, explicitly release Tier 1 with authoritative query APIs and a replayable ledger export.

Record a short working demo now. This protects your interview deadline even if later hardening takes longer.

Weeks 7-8: turn working code into evidence

19 / First release - 38 planned hours

Week 7 | 20 hours: operational visibility

Learn (3h): histograms, pprof, execution traces, saturation, and recovery time.

Build (8h): metrics, four useful dashboards, reproducible workloads, Compose, readiness/liveness, and graceful shutdown.

Verify and explain (9h): compare sync-per-order and group commit, profile a bottleneck, re-run one optimization, and test DB outage plus slow consumers under load.

Week 7 gate: the report distinguishes engine time from durable latency and HTTP latency, includes errors and saturation, and links raw results.

Week 8 | 18 hours: freeze and demonstrate

Learn and review (3h): revisit every design claim and remove anything the evidence does not support.

Build (5h): replay-inspect CLI, quickstart, architecture diagram, known-limitations page, and a 3-minute demo script.

Verify and explain (10h): clean-machine setup, crash drill, restore from copied log, money checks, retry drill, final test suite, and a mock interview.

Week 8 gate: tag v0.1 with test reports, measured results, a demo recording, and a clear Tier 1 capability list. Begin interviewing with this release.

Your weekly learning loop

Use 30-45 minutes for reading, 2-3 hours for implementation, and 45-60 minutes for tests and explanation on a typical workday. On the sixth day, review a fixture or profile and record a two-minute explanation. Leave one day off or available for recovery.

Across eight weeks, the planned 156 hours deliberately leaves 36-84 hours unallocated under the 4-5 hours/day assumption. Do not spend that reserve on additional infrastructure before the release gates pass.

Months 3-4: operate a paper-trading service

20 / Tier 2 - weeks 9-16

Week	Focus	Exit evidence
9	Add ETH/USDT under the global writer; enforce account and open-order limits.	Shared-USDT overspend test; mixed-symbol deterministic replay.
10	Versioned fees, reserve bounds, minimum amounts, and price collars.	Rounding and split-fill tests; fee account reconciles per asset.
11	Consistent snapshots and log rotation; preserve archive.	Snapshot + suffix equals full replay; snapshot pause measured.
12	Off-host backup and restore runbook; corruption drill.	Restore onto a fresh instance; record backup age and time to readiness.
13	Incremental depth protocol and resync; private order feed.	Deliberately dropped message causes successful client resync.
14	Authentication hardening, token rotation, limits, and operator halt.	Cross-user denial tests; revoke token; halt blocks new orders but permits defined cleanup.
15	Deploy paper service on a persistent-volume VM; automate startup.	48-hour soak, metrics retention, log capacity check, and resource limits.
16	Upgrade rehearsal, rollback/migration policy, and release v0.2.	Golden replay fixtures pass; compatible rollback or explicit forward-only migration demonstrated.

Budget roughly 16-20 planned hours per week plus learning and recovery buffer. Add FOK or bounded market orders only if these gates are ahead of schedule. An admin panel is optional; authenticated operator commands and runbooks are enough.

What this tier adds to your interview story

You can describe disk growth, stale reads, backup loss windows, authentication boundaries, resource limits, and safe upgrades with actual incidents or drills. This is where "works on my laptop" becomes an operated service.

Tier 2 is still single-node for authoritative writes. Recovery downtime and machine-loss exposure remain explicit limitations.

Month 5: choose one advanced question

21 / Tier 3 - weeks 17-20

Recommended track: replicate the whole state machine

Use an established Raft implementation after evaluating its documentation and storage API. Keep one global log and the same deterministic Apply function. Consensus establishes the committed prefix; acknowledge only after commit and local apply. Raft is a protocol for agreement, not a replacement for application correctness. [12]

Week	Work and experiment
17	Learn terms, leader election, majority commit, log matching, and snapshots. Document the library's persistent storage obligations.
18	Integrate three nodes on separate processes/volumes. Route mutations to the leader; only apply committed commands. Carry idempotency through the replicated state.
19	Inject leader loss, network partition, restart, and lagging follower catch-up. Reject writes without quorum. Prevent stale leaders from publishing authoritative side effects.
20	Measure outage time and recovery; verify every acknowledged operation appears once. Publish a failure matrix and release the experiment separately if unfinished.

For authoritative reads, use the library's supported leadership/read barrier rather than reading arbitrary follower memory. Make projections idempotent and side effects fenced. Three nodes on one laptop test protocols but do not provide independent machine availability.

A valid alternative: a deep performance/storage study

If consensus exceeds four weeks, compare book data structures, snapshot strategies, batch policies, and allocation behavior under fixed workloads. Publish reproducible experiments and a careful capacity model. This is a complete advanced deliverable, not a failed version of replication.

What to postpone

Independent account and symbol shards require durable reservation IDs, state transitions, deduped settlement messages, retry rules, and recovery across failures. That is a separate project. Do not combine new sharding, consensus, and Kubernetes in the same month.

Go coverage: the language through real work

22 / Learning map, part one

Concept	Use it here	Explain it without notes
Types and methods	Price, Qty, Side; checked constructors.	Why named types prevent accidental mixing; value versus pointer receiver.
Slices and arrays	Levels, batch buffers, depth copies.	len/cap, backing-array aliasing, append growth, retention by subslices.
Maps	Order index, accounts, dedupe.	Unspecified iteration; concurrency safety; canonical sorting.
Interfaces	Journal and transport seams.	Method sets, implicit satisfaction, typed nil, interface cost measured rather than assumed.
Errors and defer	I/O failures and cleanup.	Wrapping, errors.Is/As, partial success, defer execution, panic boundaries.
Packages and modules	Pure core versus I/O shell.	Import cycles, internal visibility, dependency versions, reproducible build.
Goroutines and channels	State owner, handlers, client writers.	Who starts/stops each goroutine; queue capacity; send/receive/close ownership.
Context and select	HTTP wait, shutdown, worker lifecycle.	Cancellation propagation, nil channel behavior, abandoned replies, no trading rollback.
Mutex and atomic	Hub registry, immutable snapshot pointer.	When synchronization is needed and why atomic publication requires immutable contents.
Testing and fuzzing	Rules, model comparison, corrupt input.	Fixtures, reproducible seeds, fuzz corpus, benchmark scope.

Read the language specification selectively when behavior surprises you; use it to settle questions about methods, channels, maps, and evaluation rules. [13]

Learn generics with a small typed utility only if it reduces duplication. Learn reflection by inspecting encoding/json behavior. Neither has to become a custom framework inside the exchange.

Go coverage: the runtime under the hood

23 / Learning map, part two - small experiments

Lab	Experiment and artifact
Scheduling	Compare CPU-bound and I/O-bound goroutines under different GOMAXPROCS values. Capture an execution trace and explain runnable versus blocked work.
Channels	Compare buffered/unbuffered queue behavior; read runtime/chan.go for your pinned Go version. Show that application-level ownership is distinct from lock-free internals. [2]
Escape analysis	Use go build -gcflags=all=-m=2 on a small example. Compare pointers, interfaces, and escaping buffers with -benchmem.
Garbage collection	Compare allocation-heavy versus reused-buffer workloads at the same load. Record heap, allocation rate, latency, and CPU. GOGC changes a memory/CPU trade-off; it is not a magic speed knob. [4]
Memory model	Write a small broken shared-state example, observe the race detector, then fix with a channel or mutex. Explain the happens-before edge. [1]
I/O and netpoll	Profile slow clients and blocked writes. Explain why a goroutine can wait without occupying an OS thread in every case; avoid promising all operations are nonblocking.
Filesystem durability	Compare Write, Sync, process exit, abrupt process death, and injected write errors. State which failure classes the experiment actually covers.
SQL	Inspect an order-history query plan, add a justified index, and compare rows scanned and timing. Explain transaction boundaries in projection.

Store each experiment in a small lab directory or separate notebook with code, command, observation, and explanation. Start with the runtime diagnostics guide for profiles and tracing. [3]

Topics that do not need product features

unsafe, cgo, assembly, compiler implementation, custom allocators, and kernel bypass are optional later study. Standard-library cryptography, HTTP, and SQL drivers should be reused; "from scratch" should refer to your engine and its design, not reimplementing every dependency.

Mastery test: predict the result, run the experiment, explain the mismatch, then connect the finding to an actual design choice.

Operational readiness has observable gates

24 / Security, capacity, and runbooks

Protect the public boundaries

Use TLS for a hosted service; validate user identity and ownership on every private endpoint. Keep tokens and secrets out of logs and source. For JWTs, validate signature algorithm, issuer, audience, and expiry, and define rotation/revocation. Seeded local tokens are a smaller Tier 1 starting point than a custom login system.

Bound HTTP bodies, connection timeouts, WebSocket buffers and subscription counts, command queues, open orders, and log growth. Retain dedupe keys for the Tier 1 dataset; cap admissions before exhausting memory. Later expiration needs a documented retry window and replayed expiry commands. Keep pprof/operator endpoints private; scan dependencies with govulncheck in CI. [14]

Readiness, failure, and shutdown

Readiness requires replay complete, state verified, journal writable, and trading enabled. Liveness should not restart a healthy process merely because PostgreSQL is temporarily unavailable.

Shutdown stops new admission, finishes or durably preserves admitted commands, flushes storage, and exits within a bound. HTTP handlers can finish waiting separately. Record applied and durable sequences.

Disk-full or uncertain Sync errors stop mutations. A projection outage advances lag; halt before log capacity is exhausted. Corruption prevents readiness and raises an operator error.

Initial engineering budgets, to validate

Budget	Meaning
Acknowledged data loss	Zero under the tested process-crash model; machine loss depends on replication/backup.
Recovery time	Measure at 100k then 1M commands. Choose a target after baseline; do not promise a fixed number in advance.
Backup recovery point	Document maximum backup age and archive delay; restore to a fresh volume.
Projection freshness	Set a lag threshold at baseline load; display staleness and alarm separately from money drift.

Write four runbooks: start/stop, restore, disk/corruption incident, and upgrade. Each names preconditions, commands, expected output, and when trading must remain halted.

Make it distinctive through visible evidence

25 / Your signature feature: the replay inspector

A matching engine is not a novel project category. Your implementation can stand out through unusually clear correctness and failure evidence. Make that the product experience an interviewer sees.

Tier 1: the explain command

Given an order ID or command sequence, print: normalized request, queue sequence, rule version, reserve before/after, counterparties in priority order, fill prices, ledger postings, final status, and full-state digest. Support replay to a chosen sequence and a structured state diff.

Tier 2: an interactive replay timeline

Let the viewer move through a recorded scenario and see book, balances, reserves, and trades change together. Highlight why each maker had priority and why an order was rejected. Show durable, applied, and projected watermarks with plain labels.

Three scenarios worth polishing

Lost response: submit, suppress the reply, restart, retry the same key, and show exactly one business effect.

Shared balance: one user spends USDT on two symbols; show the precise sequence and why only the affordable set is accepted.

Projection outage: stop PostgreSQL, keep a bounded durable backlog, restore it, and show checkpoint catch-up plus zero drift at the same sequence.

An evidence directory

Keep versioned workload definitions, raw benchmark output, profiles, crash reports, regression fixtures, ADRs, and demo recordings. Every screenshot should identify its commit or release. Let a reviewer reproduce the headline demonstration with one documented command.

Priority order: replay inspector first; failure laboratory second; beautiful charting third. A viewer should understand the engineering even without reading the whole repository.

Avoid claiming global uniqueness or a guaranteed interview outcome. The credible claim is that you designed, implemented, measured, and can explain this particular system.

Prepare the interview around decisions

26 / Explain the trade-offs you actually made

A three-minute demonstration

0:00-0:30 | State the scope: simulated spot exchange, one global state owner, durable journal, exact accounting.

0:30-1:15 | Place crossing orders. Explain FIFO, maker price, and reservation release in the inspector.

1:15-2:00 | Trigger the lost-response/restart scenario. Show replay and retry safety.

2:00-2:40 | Show latency boundaries and one measured optimization, including the workload and errors.

2:40-3:00 | State current limits and the next decision you would make with more load or reliability requirements.

Likely question	Your answer should point to
Why a single writer?	Atomic state ownership and simpler replay; a throughput ceiling you measure.
Is it lock-free?	No claim of formal lock-freedom. Book mutation needs no application mutex.
What if success is lost?	Durable dedupe result and same-key retry, including after restart.
How do funds stay correct?	Per-asset balanced postings plus reservation invariants, not just a zero drift chart.
How do you scale?	First measure the bottleneck; add symbols under the writer; replication for availability; sharding is a separate consistency problem.
What did Go teach you?	A specific aliasing, cancellation, allocation, or scheduling experiment.
What remains unsafe?	Explicit machine-loss, software-bug, corruption, and operational limits.

Resume wording after the evidence exists

"Built a deterministic Go spot-exchange simulator with durable command logging, restart-safe idempotency, double-entry accounting, and replay diagnostics; validated with model-based tests and injected crashes." Add measured throughput/latency only with workload and durability context.

Maintain 8 short ADRs: ownership; units; order rules; acknowledgement; WAL format; dedupe; projections; recovery/upgrades. Write them when deciding, not from memory at the end.

The release checklist and scope guard

27 / What finished means

Tier 1 release gate

- [] One symbol with documented GTC/IOC/cancel/self-trade semantics and exact fixed-point units.
- [] Reference tests and money/reservation invariants pass for recorded and generated cases.
- [] WAL errors halt mutation; partial-tail and complete-frame corruption are handled differently.
- [] Full-state replay agrees; acknowledged operations survive process crash; retries cannot duplicate effects.
- [] API ownership, queue limits, timeout behavior, and query consistency are documented and tested.
- [] Live view and ledger are inspectable; if SQL is included, its rebuild/checkpoint tests pass.
- [] Load report separates pure matching from durable and HTTP latency, including rejection/error rates.
- [] Fresh-machine quickstart, replay demo, limitations, and release tag exist.

Tier 2 additions

Shared-asset multi-symbol test; fee rounding proof/tests; snapshot + log equivalence; off-host restore; feed resync; resource limits; operator halt; security checks; soak and upgrade report.

Tier 3 additions

Documented quorum/fencing/read policy; leader-loss and partition tests; no acknowledged effect lost or duplicated within the stated failure model; recovery and latency report. Or finish the explicitly scoped performance/storage track.

If time runs short, cut in this order

Extra charts -> ticker/candles -> FOK/market orders -> incremental depth -> public deployment -> SQL read projection. Keep matching semantics, money checks, authoritative dedupe, durable replay, and a runnable demonstration.

Next action: start with the first three sessions on page 17. Do not wait to understand every runtime topic before implementing the first tested state transition.

References and review provenance

28 / Primary sources and reading order

Reviewed source inputs: CEX_MASTER_PLAN.md, sections 0-18 and Appendix A; CEX_MASTER_PLAN.pdf, 22 pages, including the architecture diagram and run-loop sketch. The PDF presents the same plan with layout differences. The older companion DOCX and a separate Go-basics repository were not supplied or reviewed.

The roadmap and numerical examples are original recommendations for your scope. Bracketed references support specific technical points, not the time estimates or a claim that the proposed implementation has been verified. Web references checked 9 September 2026.

- [1] [Go memory model](#)
- [2] [Go runtime channel implementation](#)
- [3] [Go diagnostics: profiling and tracing](#)
- [4] [Go garbage collector guide](#)
- [5] [PostgreSQL: storage reliability](#)
- [6] [PostgreSQL: asynchronous commit](#)
- [7] [Coinbase Exchange: matching engine rules](#)
- [8] [Go fuzzing tutorial](#)
- [9] [Go data race detector](#)
- [10] [k6: arrival-rate virtual-user allocation](#)
- [11] [Prometheus: histograms and summaries](#)
- [12] [Raft paper: In Search of an Understandable Consensus Algorithm](#)
- [13] [Go language specification](#)
- [14] [Go security guidance](#)

Read in the order you need it

Weeks 1-2: [7,13]. Weeks 3-4: [1,2,5,6,8,9]. Weeks 5-8: [3,10,11,14]. Months 3-4: [4] and your pinned dependency documentation. Month 5: [12]. Pair each reading with a test, experiment, or ADR.